HERALD Codebase Audit & Architecture Deep Dive

A Senior Engineer's Deconstruction

Engineering Mentor Analysis

May 21, 2026

Contents

1 Executive Summary & Core Mission

HERALD (formerly Matrix) is a fully on-premises, AI-powered phishing domain detection platform built specifically to protect Critical Sector Entities (CSEs) like banks and government portals.

1.1 The Problem Being Solved

Most commercial Threat Intelligence platforms are expensive and rely heavily on external APIs (like VirusTotal or Shodan), which creates data sovereignty and privacy concerns. Furthermore, by the time a phishing URL is reported to VirusTotal, the damage is often done.

1.2 HERALD's Solution

You designed HERALD to be entirely self-sufficient. It ingests data directly from the firehose of internet creation (Certificate Transparency logs via Certstream), runs lightweight machine learning (lexical analysis) to filter the noise, and then falls back to heavy operations (visual OCR) to confirm borderline cases. This allows HERALD to detect phishing *within minutes of registration*, completely bypassing third-party APIs.

2 Architectural Evolution

Your engineering journey on HERALD reveals a classic progression from a simple heuristic script to a complex, multi-stage, data-driven pipeline.

2.1 The Lexical Era (v3 - v6)

Initially, you relied solely on the domain name string (lexical features) to determine maliciousness.

- **v3**: Baseline lexical model (Precision: 0.877). You realized that just looking for hyphen counts and entropy wasn't enough.
- **v5**: Added Tranco legitimate domains. By injecting a massive "legitimate" class, you taught the model what a normal domain looks like, pushing precision up to 0.941.
- **v6**: Integrated WHOIS and SSL/DNS features. You realized attackers have patterns in their infrastructure (e.g., specific cheap TLDs, missing SSL).

2.2 The Lexical Ceiling & The OCR Pivot (v7)

You hit a fundamental wall at F1 ∼0.91. Why? Because attackers started registering completely random domains (e.g., `secure-update-123.com`) and hosting SBI login pages on them. Lexically, `secure-update-123.com` doesn't look like `sbi.co.in`, so your lexical model missed it.

Your clever solution (v7): A Two-Stage Pipeline. Instead of forcing the ML model to guess, you introduced a visual fallback (`cv_ocr_analyzer.py`). If the domain looks even remotely suspicious (or targets a known CSE), you fire up a headless Chrome browser, take a screenshot, and use perceptual hashing (`imagehash`) against known CSE templates (e.g., the NIC or SBI login screen). This shifted HERALD from a "domain guesser" to an "active threat hunter."

2.3 Transformer Experimentation (v8)

You tried character-level Transformers (v8) to squeeze out more performance, but the ROI wasn't there. Traditional tree-based ensembles (XGBoost + Random Forest) combined with OCR remained the most practical, resource-efficient solution. This was a mature engineering decision: knowing when to stop chasing deep learning hype in favor of operational stability.

3 End-to-End Execution Trace: The Domain Lifecycle

Let's trace exactly what happens when an attacker registers `sbi-login-secure.xyz`.

3.1 Stage 1: Ingestion (`certstream_monitor.py`)

1. The attacker registers the domain and generates an SSL certificate.
2. This certificate is broadcast to the global Certificate Transparency (CT) log.
3. `certstream_monitor.py` is listening to a WebSocket (`wss://certstream.calidog.io`).
4. The monitor receives `sbi-login-secure.xyz`.
5. It runs a fast fuzzy match (`is_suspicious()`) looking for CSE keywords (like 'sbi').
6. It flags the domain and pushes it to a Redis queue: `redis_client.rpush("domain_analysis_queue", job_data)`.

3.2 Stage 2: Orchestration & Feature Extraction

1. A background worker pops the job from Redis.
2. `domain_analyzer.py` takes over. It extracts lexical features using `lexical_features.py`:
   - Calculates Levenshtein distance to 'sbi' (very close).
   - Flags the `.xyz` TLD as highly malicious.
   - Detects suspicious trigrams ('sec', 'log').
3. It calls the WHOIS and DNS modules to check domain age (0 days old $\rightarrow$ highly suspicious).

3.3 Stage 3: ML Inference

1. The extracted feature vector is fed into the loaded v7 Ensemble Model (Random Forest + XGBoost).
2. The model outputs a probability score (e.g., 0.85).
3. Because the score is > 0.571, the domain is flagged for Stage 2 Enrichment.

3.4 Stage 4: Visual Enrichment (`cv_ocr_analyzer.py`)

1. `domain_analyzer.capture_screenshot()` uses Selenium headless Chrome to visit `sbi-login-secure.xy`
2. It takes a screenshot and saves it to `evidence/temp/`.
3. It fetches the raw HTML to check for `<form>` tags and password fields (`content_classifier.py`).
4. Perceptual hashing compares the screenshot to `data/cse_refs/sbi_login.png`. The Hamming distance is low (< 15).

3.5 Stage 5: Persistence & Monitoring

1. The final verdict is 'Phishing'.
2. The result is written to the PostgreSQL database via SQLAlchemy (`DomainScan` model).
3. If the domain was parked (no content yet), it would be written with label 'Suspected' and picked up by `monitoring/run_workers.py` (APScheduler) to be re-scanned every 24 hours for up to 90 days.

4 ML & Feature Engineering: Mitigating Attacker Behavior

Your feature engineering directly maps to psychological and technical attacker behaviors.

- **Subdomain Keyword Injection**: Attackers use `sbi.secure-update.com`. Your `brand_keyword_posit` feature specifically detects if the brand is in the subdomain vs the registered domain.
- **Cheap Infrastructure**: Attackers buy `.xyz` or `.top` domains in bulk. Your `is_malicious_gtld` feature acts as a massive penalty for these.
- **Homoglyphs**: Attackers use Cyrillic 'a' instead of Latin 'a'. You use `idna` decoding and unicode normalization to unmask these.
- **Suspicious Trigrams**: Words like 'login', 'secure', 'auth' are common in phishing. Your n-gram trigram extraction captures these semantic red flags natively.

The Threshold (0.571): You explicitly tuned the threshold to 0.571 rather than 0.5. This shows an understanding of the business context: False Positives create alert fatigue for SOC analysts, while False Negatives let threats through. 0.571 was your mathematical sweet spot for Indian CSEs.

5 Infrastructure & Operational Failure Modes

HERALD is built as a distributed microservices architecture using Docker Compose.

5.1 Subsystem Breakdown

- **API (FastAPI)**: Handles manual scan requests, dashboard data retrieval, and JWT authentication.
- **Ingestion Workers**: Scripts running continuously to feed Redis.
- **Analysis Workers**: Pop from Redis, run ML, drive Selenium.
- **Database (PostgreSQL)**: Source of truth.
- **Queue (Redis)**: The glue holding ingestion and processing together.

5.2 Failure Mode Analysis

- **Certstream WebSocket Disconnects**: `certstream` is notoriously flaky. *Your Mitigation*: You built a crt.sh HTTP polling fallback in `certstream_monitor.py` to query certificates manually if the WebSocket dies. This is a very mature, defensive programming pattern.

- **Selenium Zombie Processes**: Headless Chrome is memory-hungry and can freeze. *Risk*: If a page has a massive JS loop, Selenium hangs. You implemented a 30-second timeout, but repeated timeouts can leave zombie Chrome processes. *Refactor needed*: Ensure explicit `driver.quit()` in a `finally` block and limit max concurrent threads.
- **Redis OOM (Out of Memory)**: If workers die, Certstream will flood Redis with millions of jobs in hours, crashing the server. *Risk*: You currently don't have job TTLs or queue size limits.

6 Technical Debt & Refactor Priorities

If you were to inherit this codebase in a corporate environment, these are the areas you would flag for refactoring:

1. **Selenium Lifecycle Management**: The `setup_driver()` logic loops through hard-coded driver paths (Windows/Linux mix). This should be completely abstracted into a Docker-native Remote WebDriver (e.g., Selenium Grid or a dedicated `browserless/chrome` container).
2. **Global State in Models**: The ML models are loaded globally in memory. Under high FastAPI concurrency, this is fine, but for horizontally scaled workers, you should use a model server (like BentoML or TorchServe) rather than loading `.joblib` files in every worker thread.
3. **Authentication**: The FastApi auth uses a basic local JWT setup with a hardcoded secret in the codebase. This needs to be moved to an environment variable and ideally integrated with an external IdP (Identity Provider) for a production SOC environment.
4. **Hardcoded Constants**: CSE Keywords are hardcoded in `lexical_features.py`. They need to be moved to the PostgreSQL database so the dashboard can add/remove watched brands dynamically without requiring a code rebuild and model retraining.

7 If Rebuilding HERALD Today

If you had to rebuild this architecture from scratch today, aiming for enterprise scale:

- **Replace Redis with Kafka/RabbitMQ**: For persistent, replayable streaming of CT logs.
- **Decouple Scraping**: Move the Selenium/Puppeteer logic to an isolated, easily horizontally scaled microservice (e.g., using AWS Lambda or a fleet of lightweight Node/Puppeteer containers) to prevent memory leaks from crashing the core Python analyzer.
- **Vector Database for Logos**: Instead of static image hashes, use a Vector DB (like Milvus or Qdrant) to store CLIP embeddings of CSE logos. This would allow for much more robust visual matching against slightly altered phishing logos.

8 Learning Roadmap & Mental Models

To re-master your codebase, trace it in this specific order:

1. The Data Pipeline (The Veins): Read `herald/ingestion/certstream_monitor.py`. Understand how data enters the system and is serialized into Redis.

2. The Brain (The ML Core): Read `ml/retrain_v3.py` alongside `herald/features/lexical_features`. Watch how raw strings turn into a numeric matrix, how the Ensembles are trained, and how the 0.571 threshold is applied.

3. The Muscle (The Orchestrator): Read `herald/core/domain_analyzer.py`. This is the heaviest file. Trace how it catches the ML prediction, decides to fire up Selenium, takes a screenshot, and calculates the `imagehash`.

4. The Face (The API): Finally, read `herald/api/main.py` to see how the PostgreSQL database serves up these results to the Streamlit dashboard and external users.

8.1 Your Engineering Style

You are a pragmatic, "ship-it" engineer who values **resilience over architectural purity**.

- You use `try/except` blocks liberally to ensure the pipeline never halts on a single bad domain.
- You built fallbacks (crt.sh when Certstream fails).
- You combine fast, cheap operations (regex/lexical) with slow, expensive operations (Selenium) in a highly efficient funnel, which is a hallmark of senior systems design.